

3 – Resolução Automática de Problemas

Carlos Ramos

csr@isep.ipp.pt

Unidade Curricular: Inteligência Artificial

Aulas Teóricas

Curso: Licenciatura em Engenharia Informática (ISEP)

Resolução Automática de Problemas

- Um Problema pode ser caracterizado como uma questão que nos é colocada de como passar de uma situação num dado mundo para outra situação pretendida, atendendo a um conjunto de regras e restrições que nos são impostas

- Mundo do Problema

- Situação (estado)

- Qual é a situação pretendida ?

- Alternativas de resolução ?

- Restrições, Regras

Resolução Automática de Problemas

- Abordagens principais à Resolução Automática de Problemas e Pesquisa Automática:
  - Inteligência Artificial
  - Investigação Operacional

- Tipos de Soluções Pretendidas:
  - Melhor Solução
  - Solução Rápida ou em tempo útil (geralmente com heurísticas)

- O problema
  - Pode não ter solução
    - Como detetar?
    - Excesso de Restrições
  - Ter muitas soluções
    - Como escolher?
    - Explosão combinatória

Resolução Automática de Problemas

- O que será então necessário para que possamos pensar em resolver um problema automaticamente?
  - Defini-lo de um modo preciso
  - Analisá-lo
  - Escolher as melhores técnicas a aplicar

Princípios de Pesquisa

- Por vezes na pesquisa automática usam-se princípios que devem ser atendidos de um modo sistemático
- Esses princípios não dão garantias de se encontrar a melhor solução, mas devem ser estudados para se poder ganhar alguma sensibilidade para o problema
- Iremos considerar os seguintes 2 princípios:
  - Primeiro em Profundidade
  - Primeiro em Largura

Gerar todas as soluções e escolher a melhor

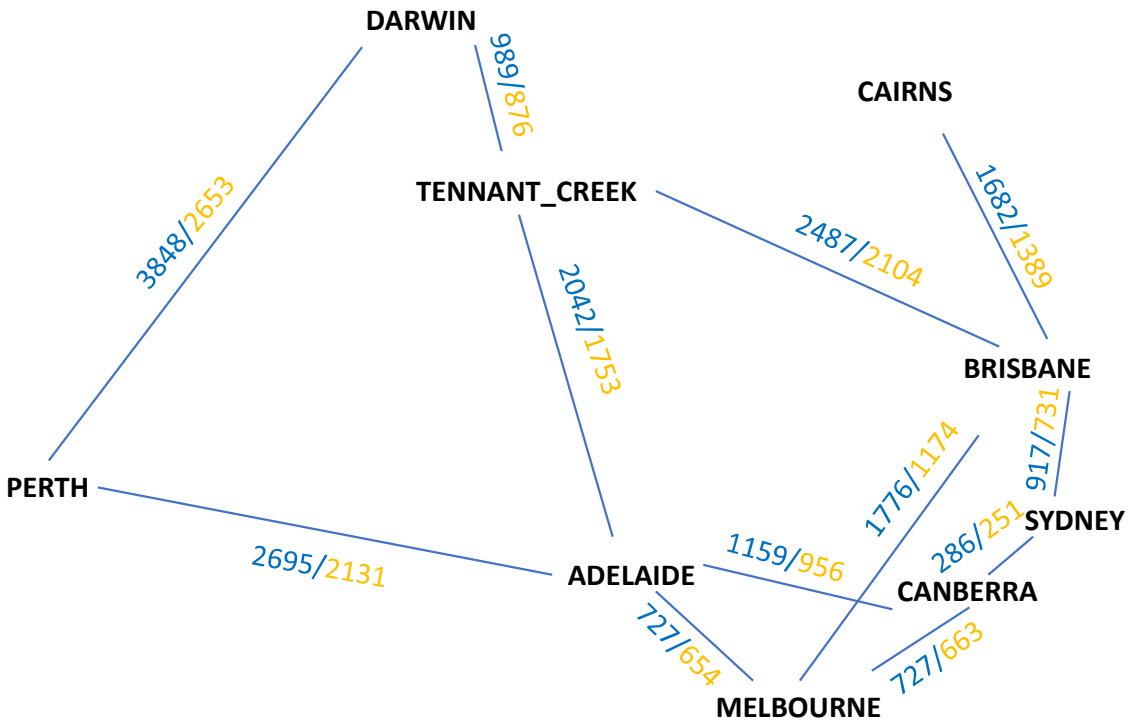
- Nos princípios de pesquisa que vamos estudar podemos assumir que acabamos quando é encontrada a primeira solução. Contudo, o princípio pode continuar a ser aplicado até que todas as soluções tenham sido geradas ou até não fazer sentido gerar mais soluções por elas serem garantidamente piores
- Podemos então selecionar a melhor solução entre todas as disponíveis
- Isso conduz a um método que garante a melhor solução, o problema é que com o aumento da dimensão do problema gerar todas as soluções é impraticável em tempo útil!

Primeiro em Profundidade

- A ideia que está inerente ao método Primeiro em Profundidade é a de tentar avançar de estado para estado até que se encontre a solução
- É um método adequado se as opções tomadas forem na direção correta, mas pode ser inadequado se a direção tomada for desadequada
- O método Primeiro em Profundidade apresenta a vantagem de ter poucos requisitos em termos de memória
- O método é também adequado para problemas que tenham várias soluções pois nesses casos aumenta a possibilidade de se optar por um caminho adequado
- Dada a possibilidade de escolher caminhos que não levem ao estado final pretendido é sempre aconselhável implementar o método com retrocesso, ou seja, se chegar a um "beco sem saída" volta atrás e explora outra alternativa
- É um método não-informado
- É o mecanismo base usado na pesquisa na linguagem Prolog
- Um dos problemas, sobretudo em grafos de grande dimensão, é seguir um caminho algo "aleatório" e visitar muito mais nós do que o necessário para chegar à solução. Isso resolve-se impondo níveis de corte (não ir abaixo de um determinado nível) mas pode implicar não encontrar solução

Um exemplo: Auto-Estradas da Austrália

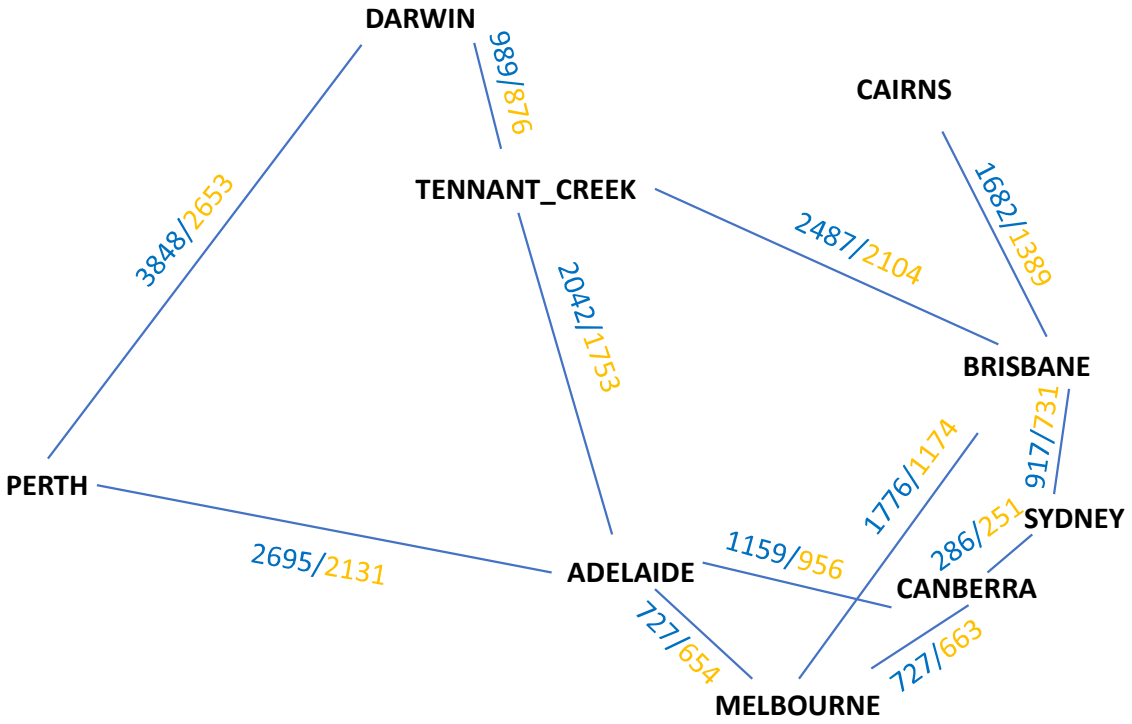


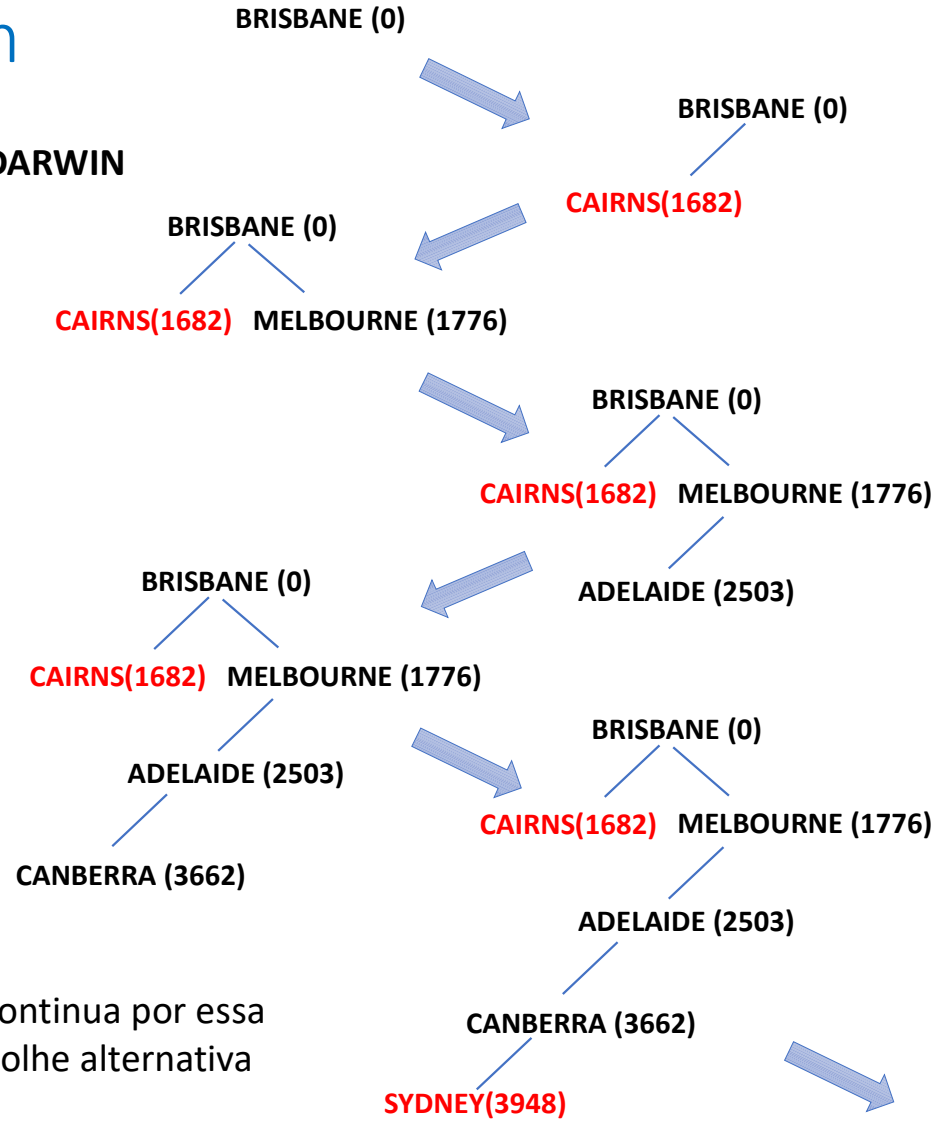


Distância por estrada/Distância menor

Primeiro em Profundidade com Retrocesso

Objetivo: ir de BRISBANE para DARWIN

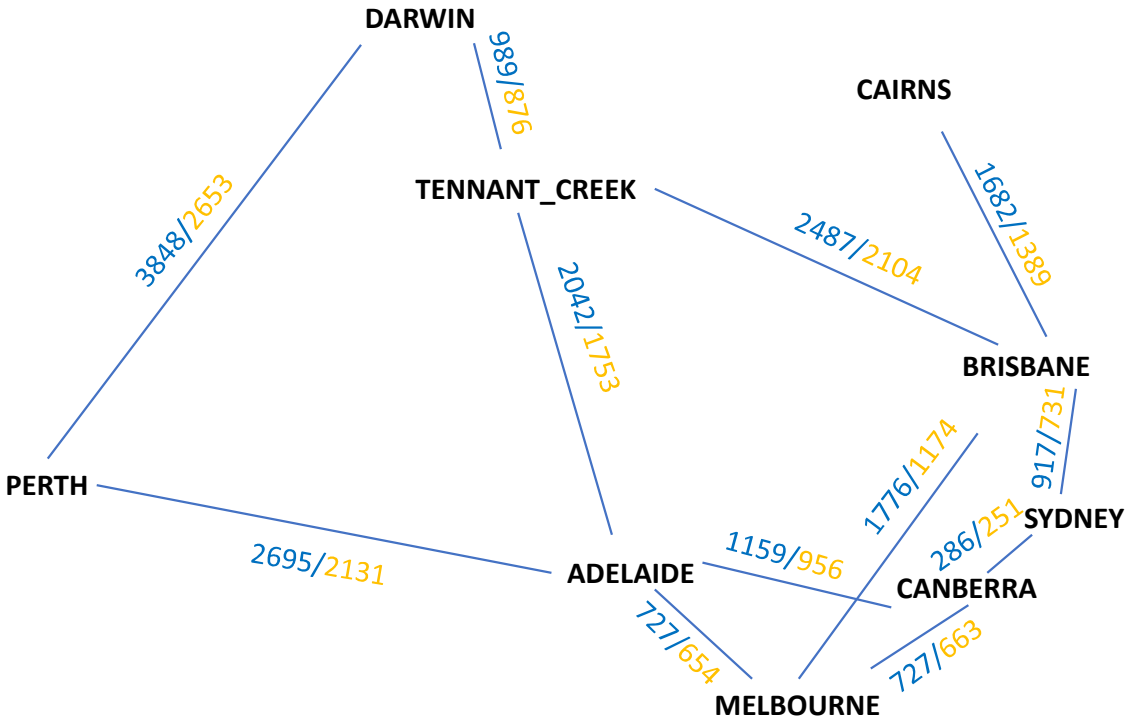


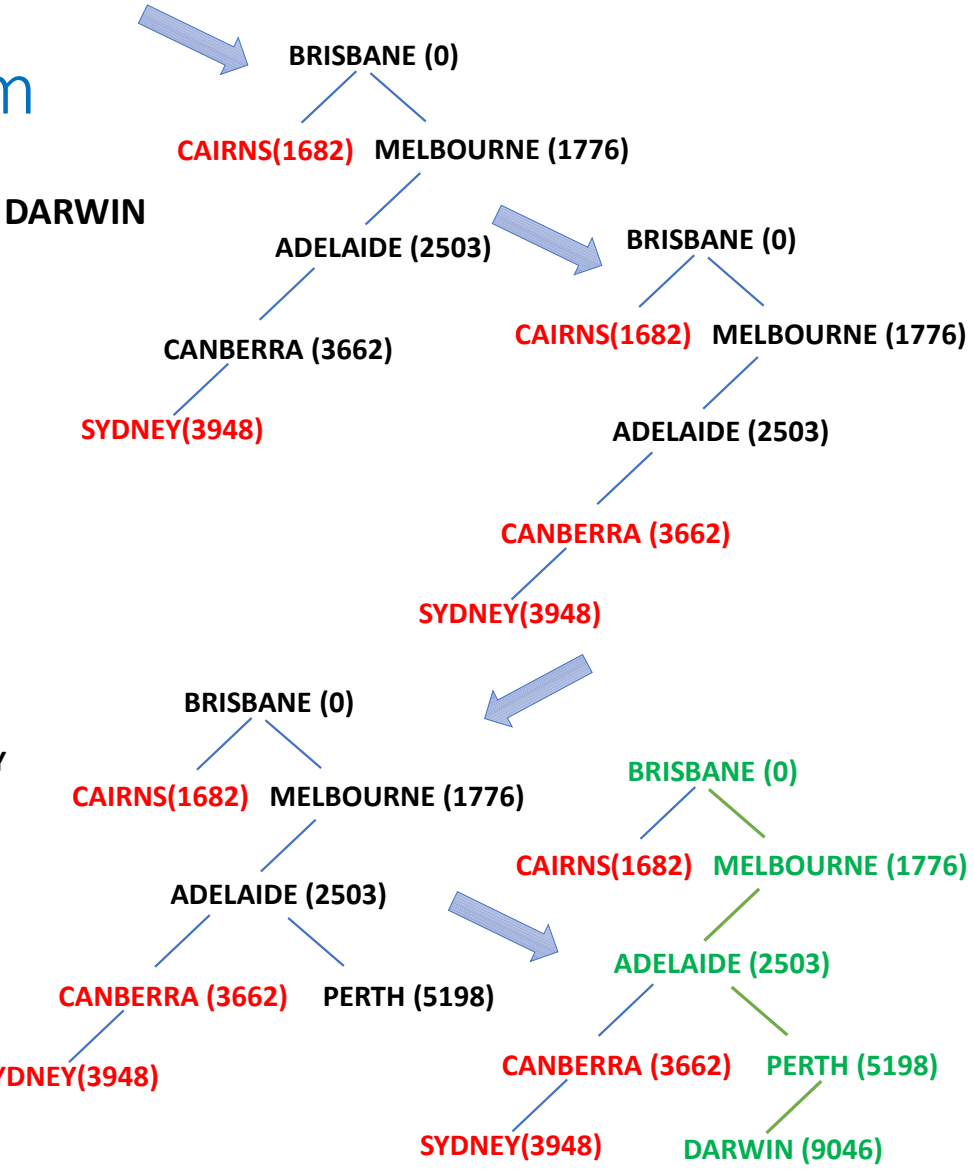


Escolher uma cidade conectada (por ordem alfabética no exemplo) e continua por essa
Se não tiver saída ou se só puder ir por onde já passou retrocede e escolhe alternativa
Evitar ir para uma cidade por onde já passou (evitar ciclos)

Primeiro em Profundidade com Retrocesso

Objetivo: ir de BRISBANE para DARWIN

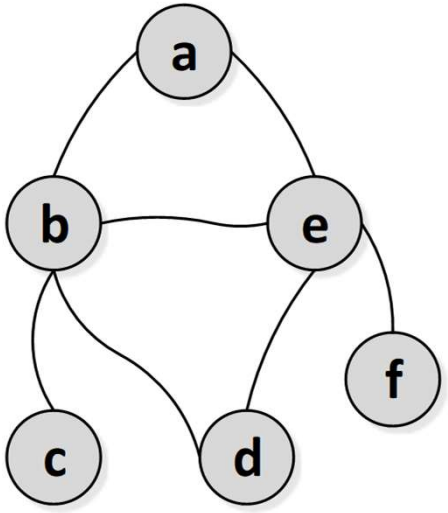




Se houver solução o método encontra solução ?
Encontra melhor solução ?
O que acontece quando temos uma dimensão muito elevada ?

Implementação do Primeiro em Profundidade Depth First Search (DFS)

Grafo Exemplo




```
%edge(Node1,Node2)
edge(a,b).
edge(a,e).
edge(b,c).
edge(b,d).
edge(b,e).
edge(d,e).
edge(e,f).

% ligacoes bidireccionais
connect(X,Y):-
        edge(X,Y);edge(Y,X).
```

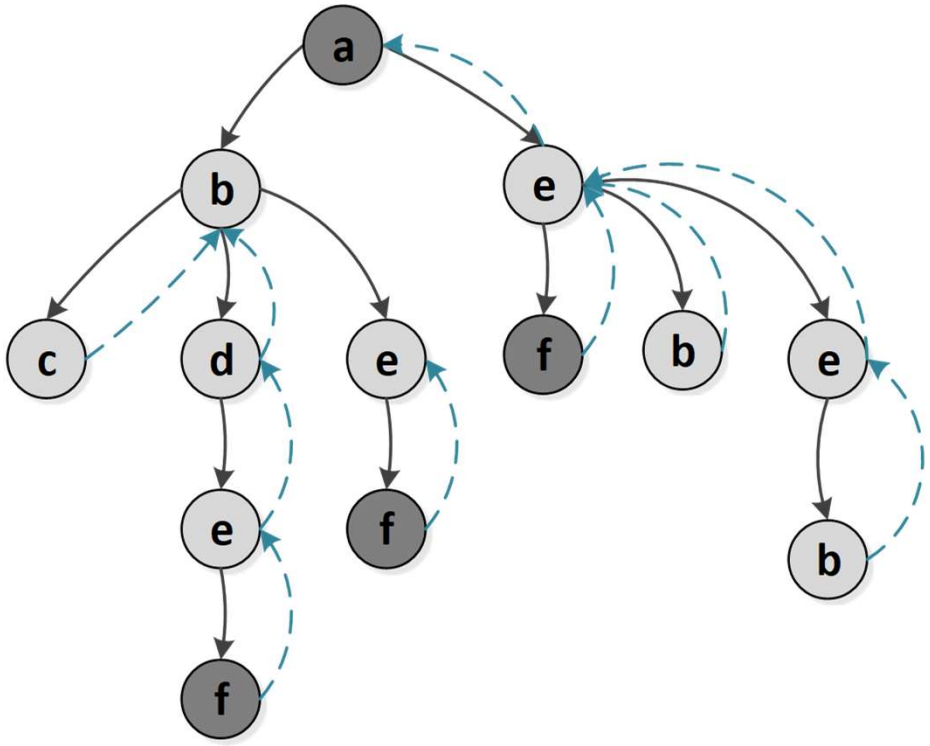
Implementação do Primeiro em Profundidade
Depth First Search (DFS)
Implementação Prolog / Soluções

```
dfs(Orig,Dest,Cam):-
        dfs2(Orig,Dest,[Orig],Cam).

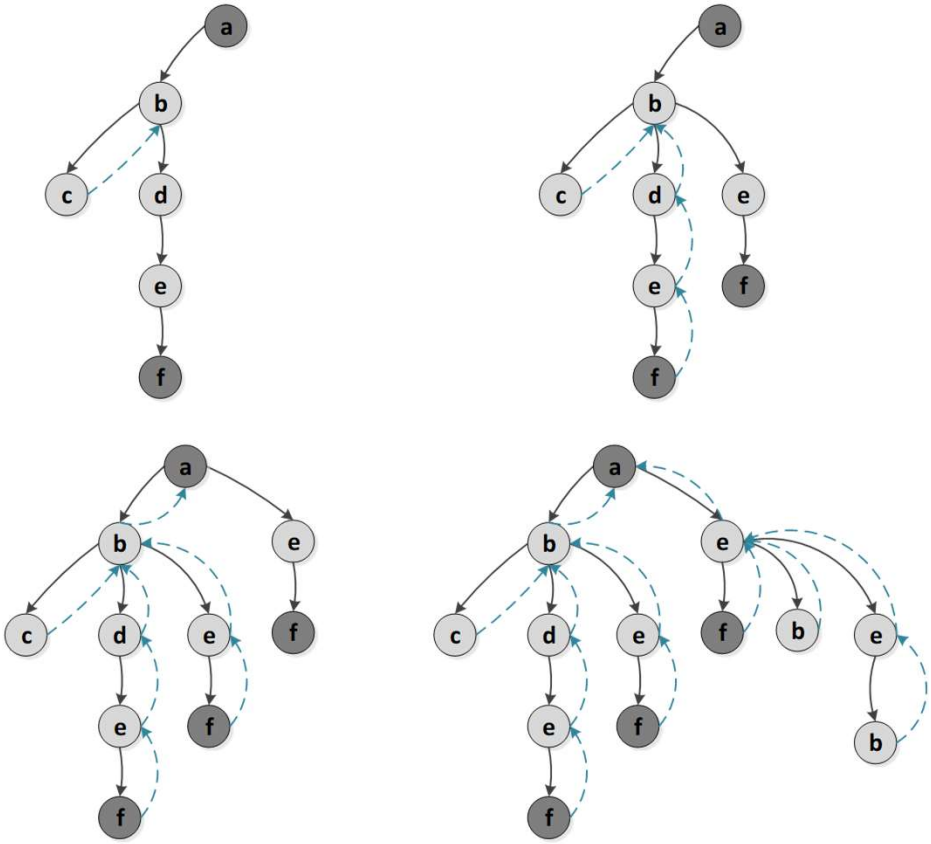
%condicao final: nodo actual = destino
dfs2(Dest,Dest,LA,Cam):-
        %caminho actual esta invertido
        reverse(LA,Cam).

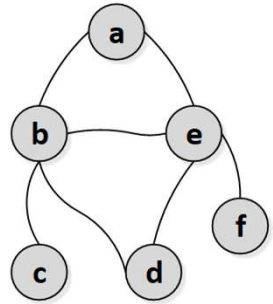
dfs2(Act,Dest,LA,Cam):-
        %testar ligacao entre ponto
        %actual e um qualquer X
        connect(Act,X),

        %testar nao circularidade p/ nao
        %visitar nodos ja visitados
        \+ member(X,LA),
        %chamada recursiva
        dfs2(X,Dest,[X|LA],Cam).
```



Implementação do Primeiro em Profundidade
Depth First Search (DFS)
Espaço do problema / Soluções






```
%edge(Node1,Node2)
edge(a,b).
edge(a,e).
edge(b,c).
edge(b,d).
edge(b,e).
edge(d,e).
edge(e,f).

% ligacoes bidireccionais
connect(X,Y):-
        edge(X,Y);edge(Y,X).
```

Depth First Search (DFS)
Implementação Prolog / Soluções

```
dfs(Orig,Dest,Cam):-
        dfs(Orig,Dest,[Orig],Cam).

dfs(Dest,Dest,LA,Cam):-
        reverse(LA,Cam).

dfs(Act,Dest,LA,Cam):-
        write(LA),nl,
        connect(Act,X),
        \+ member(X,LA),
        dfs(X,Dest,[X|LA],Cam).
```

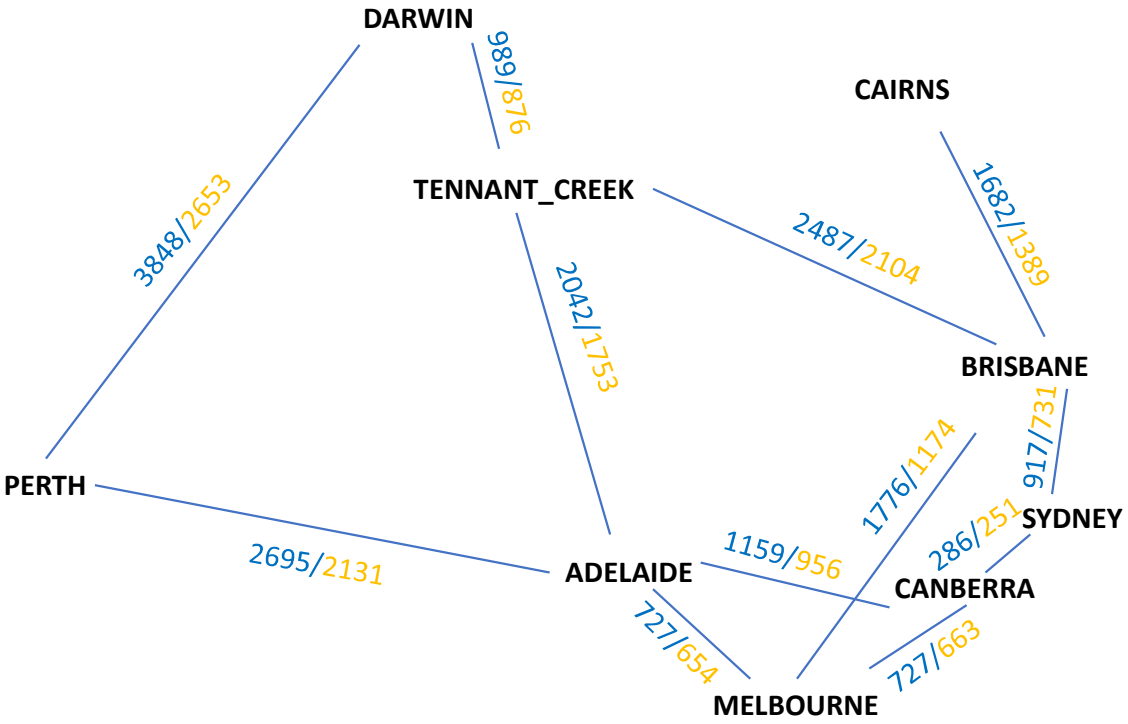
```
?- dfs(a,f,Caminho).
[a]
[b,a]
[b,a]
[d,b,a]
[e,d,b,a]
[f,e,d,b,a]
Caminho = [a, b, d, e, f] ;
[b,a]
[e,b,a]
[f,e,b,a]
Caminho = [a, b, e, f] ;
[e,b,a]
[a]
[e,a]
[f,e,a]
Caminho = [a, e, f] ;
[e,a]
[b,e,a]
[b,e,a]
[e,a]
[d,e,a]
[b,d,e,a]
false.
```

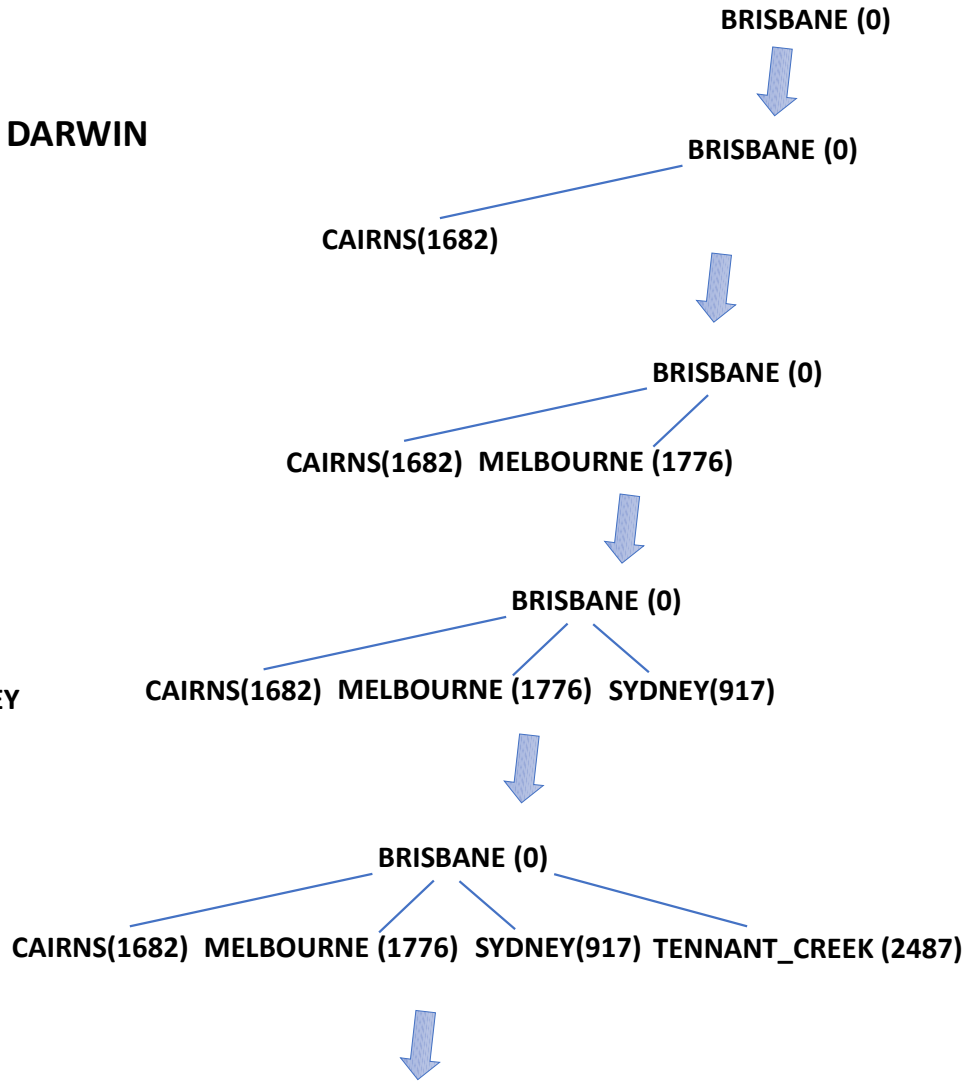
Primeiro em Largura

- Segundo este método só irá ser possível efetuar a pesquisa no nível N da árvore se todos os nós do nível N-1 tiverem sido explorados
- Pode-se dizer que a árvore de pesquisa em construção é explorada transversalmente, derivando daí o nome do método
- Garante a obtenção da solução ao nível mais próximo da raiz, o que não quer dizer que seja a melhor solução
- Pode requerer muita memória e espaço e tempo para problemas mais complexos
- É um método não-informado
- Para grafos de grande dimensão, onde o ponto de origem e destino estão separados por muitos nós intermédios (níveis) o método é ineficiente

Primeiro em Largura

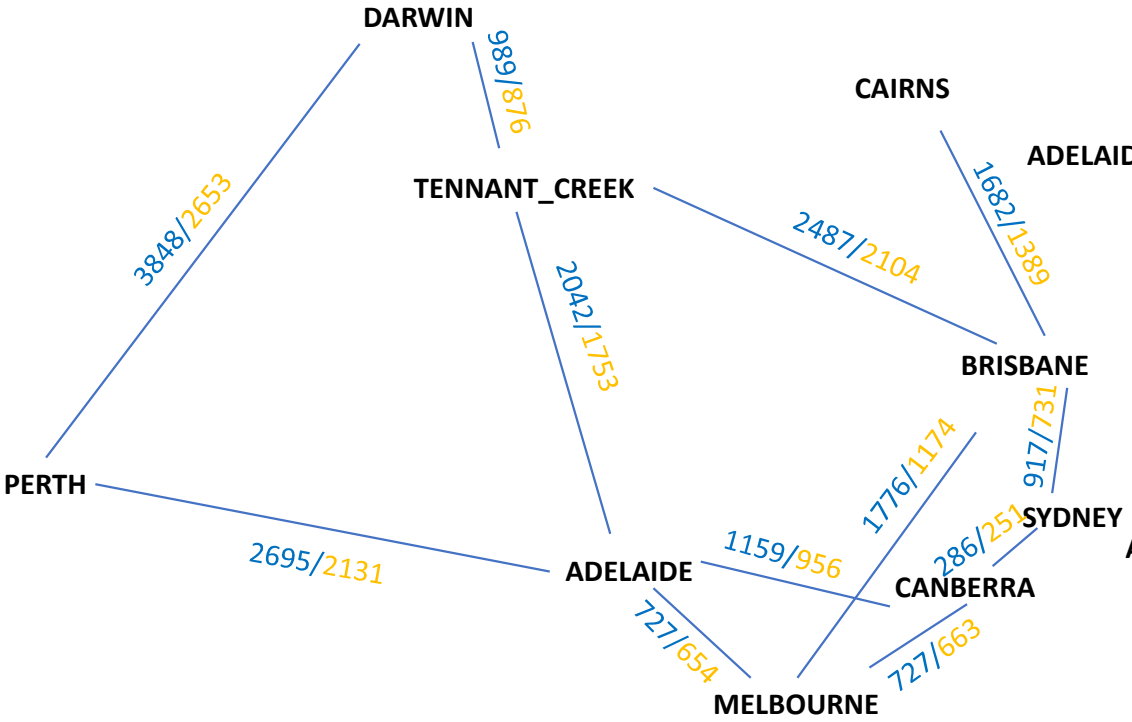
Objetivo: ir de BRISBANE para DARWIN

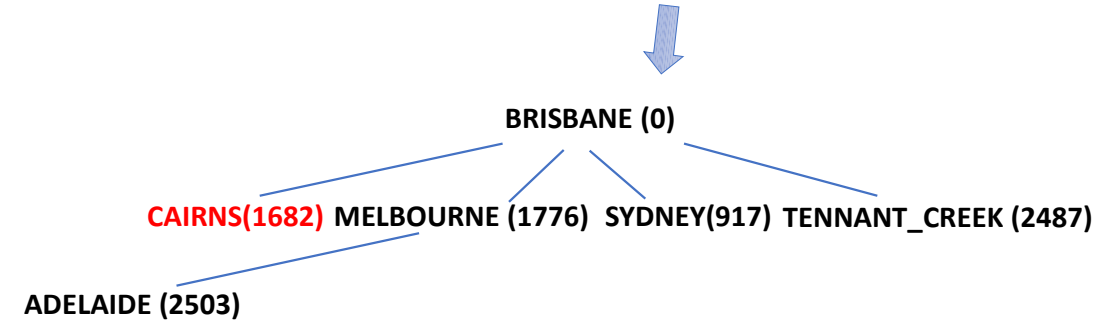


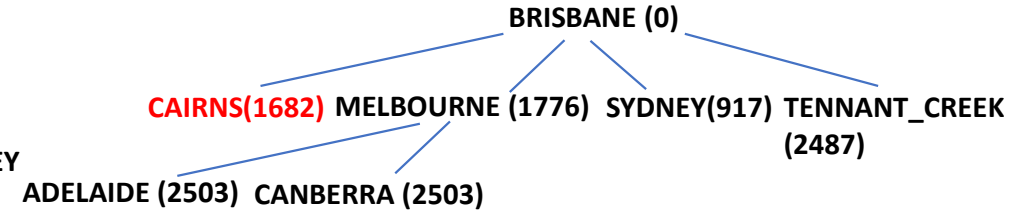


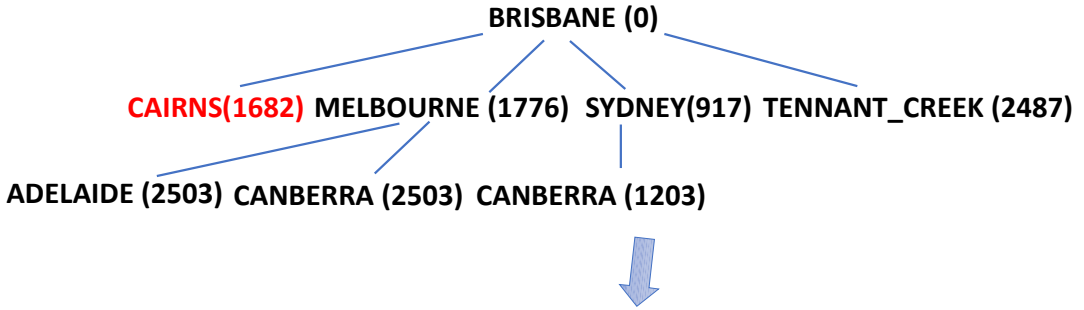
Só avança para o nível N+1 quando o nível N está preenchido

Primeiro em Largura

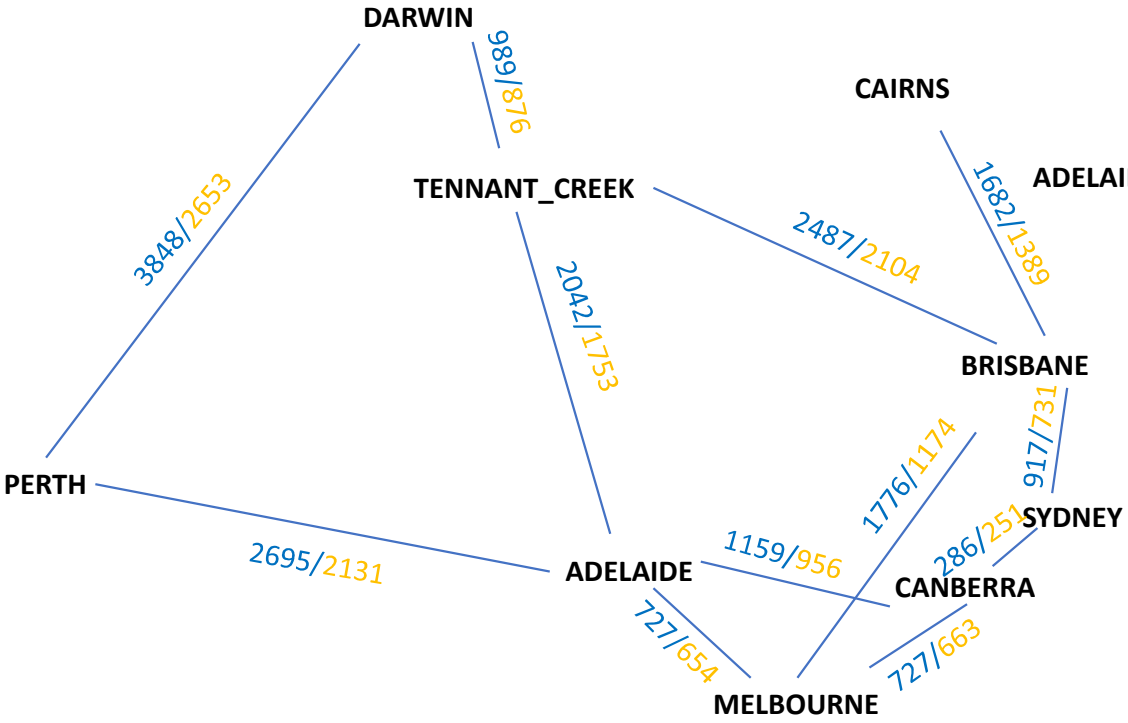


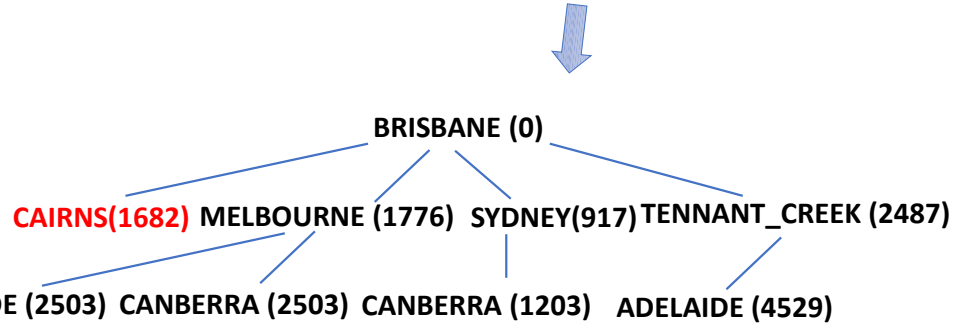


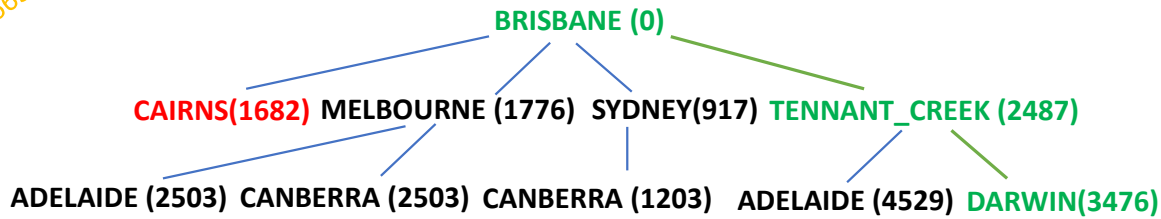




Primeiro em Largura

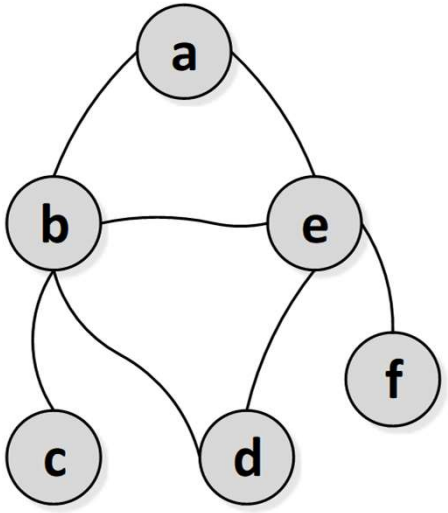






Se houver solução o método encontra solução ?
Encontra a solução ao nível mais próximo da raiz
Encontra a melhor solução ?
O que acontece quando temos uma dimensão muito elevada e a origem e destino a muitos níveis de distância ?

Implementação do Primeiro em Largura
Breadth First Search (BFS)
Grafo Exemplo




```
%edge(Node1,Node2)
edge(a,b).
edge(a,e).
edge(b,c).
edge(b,d).
edge(b,e).
edge(d,e).
edge(e,f).

% ligacoes bidireccionais
connect(X,Y):-
        edge(X,Y);edge(Y,X).
```

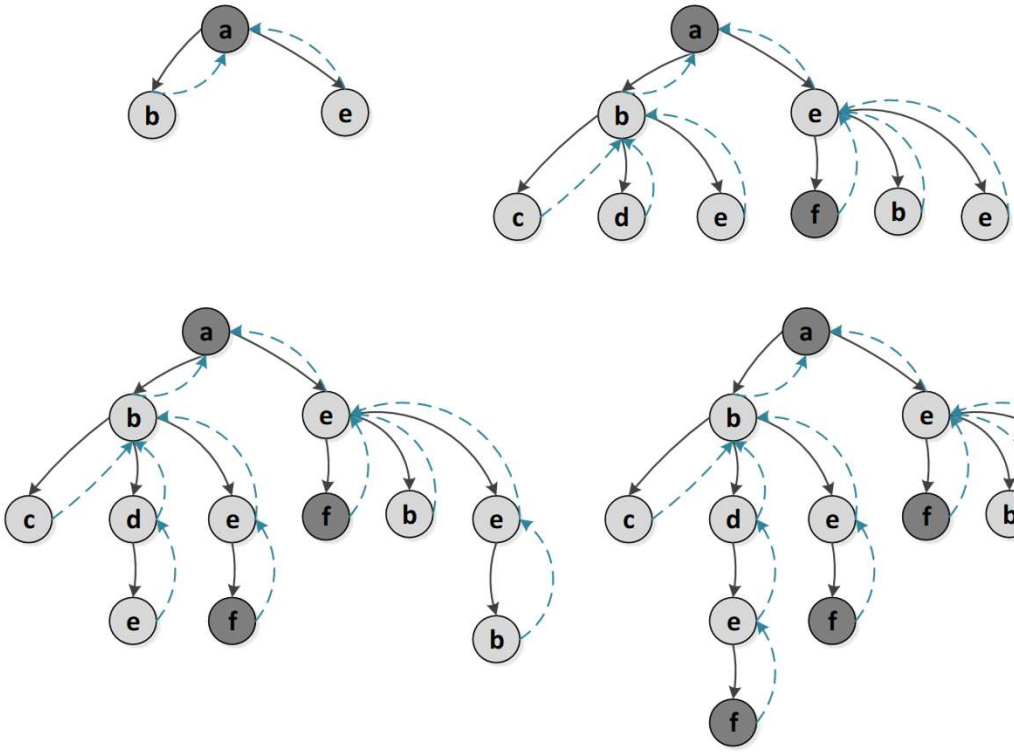
Implementação do Primeiro em Largura
Breadth First Search (BFS)
Implementação Prolog

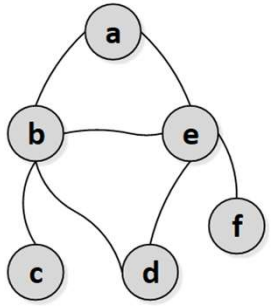
```
bfs(Orig,Dest,Cam):-bfs2(Dest,[[Orig]],Cam).

%condicao final: destino = nó à cabeça do caminho actual
bfs2(Dest,[[Dest|T]|_],Cam):-
        %caminho actual está invertido
        reverse([Dest|T],Cam).

bfs2(Dest,[LA|Outros],Cam):-
        LA=[Act|_],
        %calcular todos os nós adjacentes nao visitados e
        %gerar um caminho novo c/ cada nó e caminho actual
        findall([X|LA],
                (Dest\==Act,connect(Act,X),\+ member(X,LA)),
                Novos),
        %novos caminhos são colocados no final da lista
        %p/ posterior exploracao
        append(Outros,Novos,Todos),
        %chamada recursiva
        bfs2(Dest,Todos,Cam).
```

Implementação do Primeiro em Largura
Breadth First Search (BFS)
Espaço do Problema / Soluções






```
%edge(Node1,Node2)
edge(a,b).
edge(a,e).
edge(b,c).
edge(b,d).
edge(b,e).
edge(d,e).
edge(e,f).

% ligacoes bidireccionais
connect(X,Y):-
        edge(X,Y);edge(Y,X).
```

Breadth First Search (BFS)
Implementação Prolog / Soluções

```
bfs(Orig,Dest,Cam):-
bfs2(Dest,[[Orig]],Cam).

bfs2(Dest,[[Dest|T]|_],Cam):-
        reverse([Dest|T],Cam).

bfs2(Dest,[LA|Outros],Cam):-
          write([LA|Outros]),nl,
           LA=[Act|_],
findall([X|LA],
(Dest\==Act,connect(Act,X),\+member(X,LA)),
Novos),
        append(Outros,Novos,Todos),
        bfs2(Dest,Todos,Cam).
```

```
?- bfs(a,f,Caminho).
[[a]]
[[b,a],[e,a]]
[[e,a],[c,b,a],[d,b,a],[e,b,a]]
[[c,b,a],[d,b,a],[e,b,a],[f,e,a],[b,e,a],[d,e,a]]
[[d,b,a],[e,b,a],[f,e,a],[b,e,a],[d,e,a]]
[[e,b,a],[f,e,a],[b,e,a],[d,e,a],[e,d,b,a]]
Caminho = [a, e, f] ;
[[f,e,a],[b,e,a],[d,e,a],[e,d,b,a],[f,e,b,a],[d,e,b,a]]
[[b,e,a],[d,e,a],[e,d,b,a],[f,e,b,a],[d,e,b,a]]
[[d,e,a],[e,d,b,a],[f,e,b,a],[d,e,b,a],[c,b,e,a],[d,b,e,a]]
[[e,d,b,a],[f,e,b,a],[d,e,b,a],[c,b,e,a],[d,b,e,a],[b,d,e,a]]
Caminho = [a, b, e, f] ;
[[f,e,b,a],[d,e,b,a],[c,b,e,a],[d,b,e,a],[b,d,e,a],[f,e,d,b,a]]
[[d,e,b,a],[c,b,e,a],[d,b,e,a],[b,d,e,a],[f,e,d,b,a]]
[[c,b,e,a],[d,b,e,a],[b,d,e,a],[f,e,d,b,a]]
[[d,b,e,a],[b,d,e,a],[f,e,d,b,a]]
[[b,d,e,a],[f,e,d,b,a]]
Caminho = [a, b, d, e, f] ;
[[f,e,d,b,a],[c,b,d,e,a]]
[[c,b,d,e,a]]
false.
```

O que acontece se trocarmos a ordem dos dois primeiros argumento do append?

```
bfs(Orig,Dest,Cam):-bfs2(Dest,[[Orig]],Cam).

%condicao final: destino = nó à cabeça do caminho actual
bfs2(Dest,[[Dest|T]|_],Cam):-
        %caminho actual está invertido
        reverse([Dest|T],Cam).

bfs2(Dest,[LA|Outros],Cam):-
        LA=[Act|_],
        %calcular todos os nós adjacentes nao visitados e
        %gerar um caminho novo c/ cada nó e caminho actual
        findall([X|LA],
                (Dest\==Act,connect(Act,X),\+ member(X,LA)),
                Novos),
        %novos caminhos são colocados no inicio da lista
        %p/ exploracao imediata
        append(Novos, Outros, Todos),
        %append(Outros, Novos, Todos),
        %chamada recursiva
        bfs2(Dest,Todos,Cam).
```